



Fullstack applications

Last reviewed: 15 days ago

Fullstack web applications combine frontend and backend technologies to deliver complete, dynamic user experiences. These applications rely on a broad technology stack covering user interfaces, backend services, databases, integrations, and increasingly, AI-driven features to function seamlessly and scale reliably.

On the frontend, developers typically use HTML, CSS, and JavaScript, often alongside frameworks like React, Next.js, or Angular. These tools provide the structure and interactivity needed for modern user interfaces,

helping manage state, render dynamic content, personalize experiences, and optimize performance across devices.

On the backend, server-side code handles tasks like processing requests, running business logic, authenticating users, integrating AI models, and interacting with databases. Developers build these services using languages like JavaScript, Python, or Java, supported by frameworks that simplify routing, middleware, and API creation.

Databases are critical in the stack, storing and retrieving application data. Relational databases like MySQL, PostgreSQL, and SQLite manage structured data and enforce data integrity, while NoSQL options like MongoDB or Cassandra offer flexibility for handling unstructured or large-scale datasets.

Modern fullstack development increasingly incorporates external services, APIs, pre-built components, and AI capabilities. This approach reduces the need to create complex features from scratch, such

as content moderation, personalized recommendations, and semantic search. As a result, development teams can build applications more quickly and efficiently.

Cloudflare's Developer Platform combines all these capabilities into a unified, globally distributed environment, offering developers everything they need to build, deploy, and scale modern fullstack applications with minimal operational overhead.

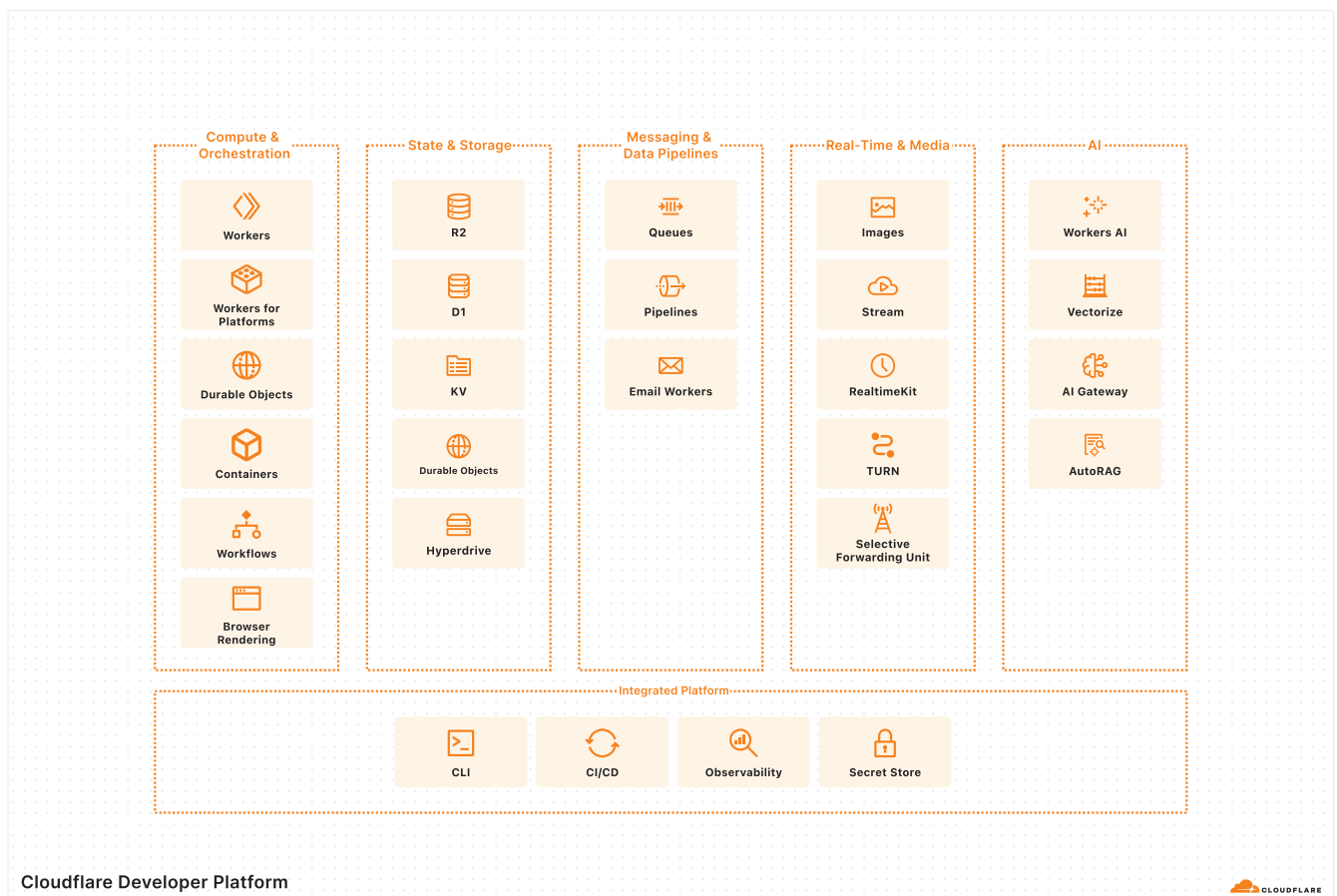


Figure 1: Cloudflare Developer Platform

Cloudflare's platform doesn't just offer individual services. Rather, it offers a **composable ecosystem**, enabling teams to build powerful applications quickly, scale seamlessly, and innovate faster without the overhead of managing infrastructure.

Fullstack application diagram

In this section, we'll present a practical example of how these services come together in a real fullstack application architecture.

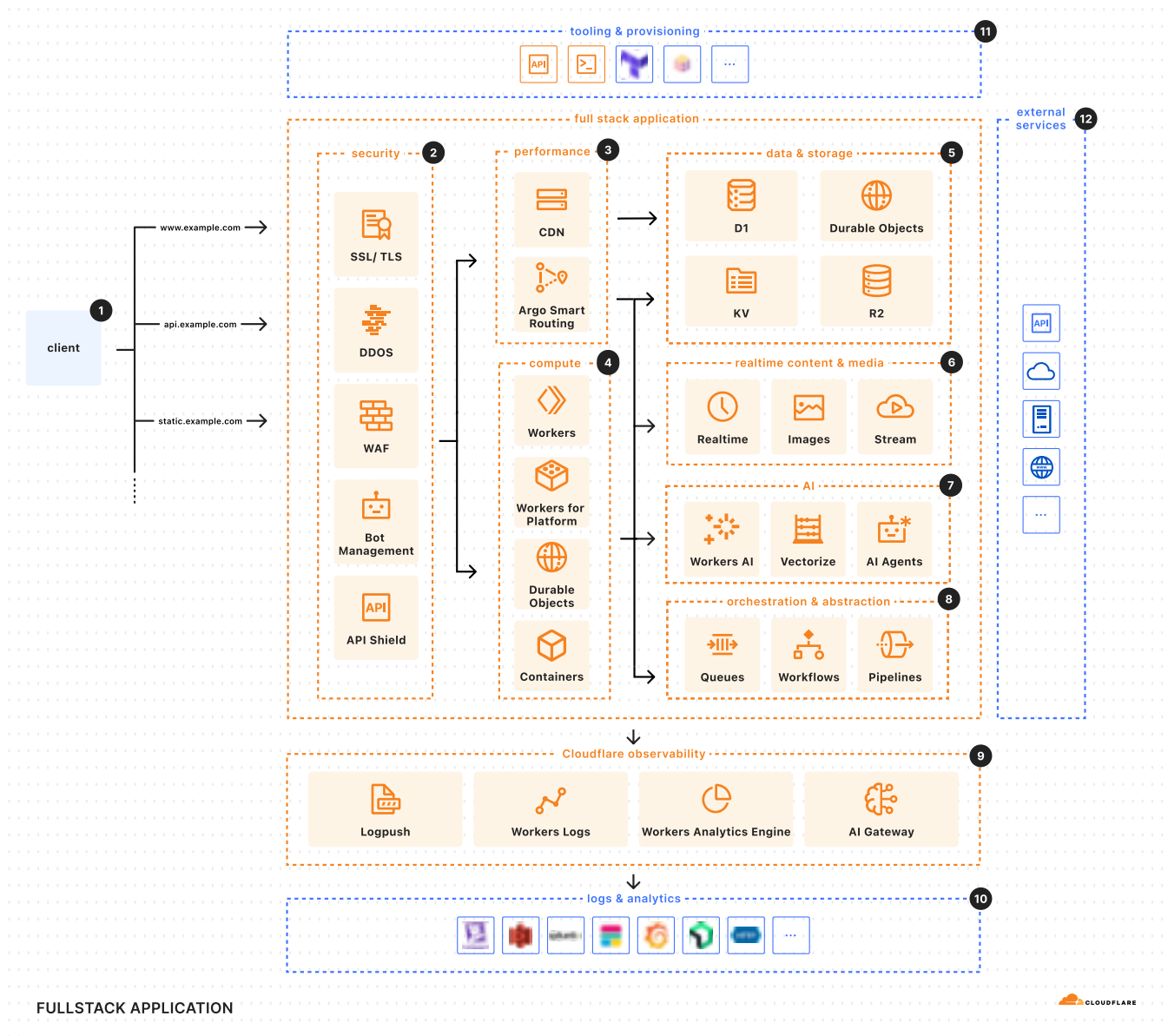


Figure 2: Fullstack application

1. Client

Sends requests to the server. This could be through a desktop or mobile browser, or native or mobile app.

2. Security

Process incoming requests to ensure the security of an application. This includes encryption of traffic using [SSL/TLS](#), offering [DDOS protection](#), filtering malicious traffic through a [web application firewall \(WAF\)](#), [mitigations against automated bots](#), and [API Shield](#) to identify and address your API vulnerabilities.

Depending on the configuration, requests can be blocked, logged, or allowed based on a diverse set of parameters. Sensible fully managed and default configurations can be used to reduce attack surfaces with little to no overhead.

3. Performance

Serve static requests from [global cache \(CDN\)](#). This reduces latency and lowers resource utilization, as the requests are being served from cache instead of requiring a request to storage & media services or compute services. Take advantage of [Argo Smart Routing](#) to route requests across the most efficient network path, avoiding congestion.

4. Compute

Process dynamic requests using serverless compute with [Workers](#). This could include authentication, routing, middleware, database interactions, and serving APIs. Moreover, [Workers Assets](#) can be used to serve client-side or server-side rendering web frameworks such as React, Next.js, or Angular. Utilize [Workers for Platforms](#) to allow users to deploy custom code on your platform or enable them to deploy their own code directly. For stateful workloads, [Durable Objects](#) provide low-latency, stateful compute by running logic close to where the object's data is stored, enabling coordination, persistence, and real-time communication at the edge.

For workloads that require the flexibility of traditional containerization, [Containers](#) allows you to run existing Docker-compatible applications on Cloudflare's global network. Containers is designed for applications needing more resources than a standard Worker.

5. Data & Storage

Introduce state to applications by persisting and retrieving data. This includes [R2](#) for object storage, [D1](#) for relational data, [KV](#) for data with high read requirements and [Durable Objects](#) for strongly consistent data storage. The [storage options guide](#) can help to assess which storage option is the most suitable for a given use case.

6. Realtime content & Media

Build real-time serverless video, audio, and data applications with [Realtime](#). Serve optimized images from [Images](#) and on-demand videos as well as live streams from [Stream](#).

7. AI

With [Workers AI](#), developers can run popular open-source models for tasks like text generation, image analysis, and content moderation powered by

serverless GPUs. [Vectorize](#) is a globally distributed vector database for similarity search, personalization, and recommendation features. [Agents](#) further extend AI capabilities - Cloudflare provides the Agents SDK that lets you build and deploy AI-powered agents that can perform tasks, interact in real time, call models, manage state, run workflows, query data, and integrate human-in-the-loop actions.

8. Orchestration & Abstraction

[Queues](#) enable durable, asynchronous messaging to decouple services and handle traffic spikes. [Workflows](#) orchestrate complex processes across APIs, services, and human approvals, abstracting away infrastructure and state management. [Pipelines](#) let you ingest high volumes of real time data, without managing any infrastructure.

9. Cloudflare Observability

Send logs from all services with [Logpush](#), gather insights with [Workers Logs](#) directly in the Cloudflare dashboard, collect custom metrics from Workers using [Workers Analytics Engine](#), or observe and control AI applications with [AI Gateway](#).

10. External Logs & Analytics

Integrate Cloudflare's observability solutions with your existing third-party solutions. Logpush supports many [destinations](#) to push logs to for storage and further analysis. Also, Cloudflare analytics can be [integrated with analytics solutions](#). The [GraphQL Analytics API](#) allows for flexible queries and integrations.

11. Tooling & Provisioning

Define and manage resources and configuration using third-party tools and frameworks such as [Terraform](#) and [Pulumi](#), Cloudflare's Developer Platform command-line interface (CLI) [Wrangler](#), or the [Cloudflare API](#). All of these tools can be used either for

manual provisioning, or automated as part of CI/CD pipelines.

12. External Service Integrations

Cloudflare's Developer Platform is built for seamless [integration with external services](#). Whether connecting to third-party APIs, databases, SaaS platforms, or cloud providers, developers can easily make outbound requests from Workers, trigger workflows based on external events, and securely exchange data across systems.

Was this helpful?



Previous

Next

A/B-testing using
Workers

Programmable
Platforms



Last updated: Oct 7, 2025

Resources

[API](#)

[New to Cloudflare?](#)

[Directory](#)

[Sponsorships](#)

[Open Source](#)

Support

[Help Center](#)

[System Status](#)

[Compliance](#)

[GDPR](#)

Company

[cloudflare.com](#)

[Our team](#)

[Careers](#)

Tools

[Cloudflare Radar](#)

[Speed Test](#)

[Is BGP Safe Yet?](#)

[RPKI Toolkit](#)

[Certificate Transparency](#)

Community

 [X](#)

 [Discord](#)

 [YouTube](#)

 [GitHub](#)

© 2025 Cloudflare, Inc. • Privacy Policy • Terms of Use

- Report Security Issues
- Trademark
-  Cookie Preferences